

KDMS 변환 규칙 매핑표

KDMS-D07 · v0.1 · 2026-10-06

MS-SQL 2019 → PostgreSQL 16 · 기본 규칙 kdms-rules.yml · 2단계(스키마 변환) 결과

자료형 (types)

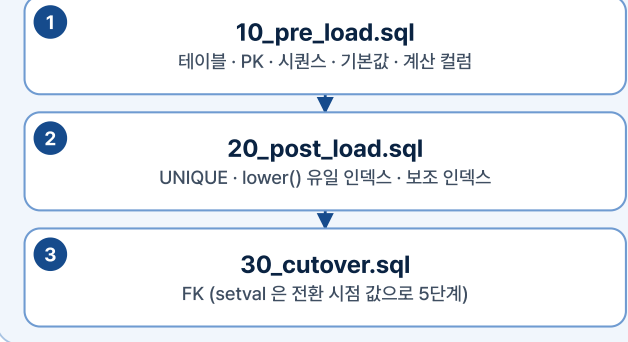
원천 타입 → 대상 타입 · 괄호는 KIS 함정 번호

datetime 3.33ms 단위	→	timestamp(3) .997 값 그대로 (A05)	(n)varchar(n) UTF-16 · CP949	→	varchar(n) UTF-8 바이트 증가 (A03)
datetime2(7) 100ns 단위	→	timestamp(6) 7→6자리 반올림 (A06)	char(n) 채움 공백	→	char(n) 검증은 RTRIM (A10)
money 8바이트 고정 소수	→	numeric(19,4) 합계 오버플로 방지 (A04)	uniqueidentifier GUID	→	uuid 정렬 순서가 달라짐
bit 0 · 1	→	boolean 앱 SQL '1' 주의 (A07)	rowversion 행 버전	→	bytea 낙관적 잠금이면 별도 설계
tinyint 0 ~ 255	→	smallint CHECK 0 ~ 255	datetimeoffset 시간대 포함	→	timestamptz(6) 오프셋 버리고 UTC 저장

⚠ 규칙 없는 타입(geography · hierarchyid · sql_variant 등)은 kdms plan 이 오류로 멈춘다

DDL 세 단계

한 번에 만들지 않고 시점을 나눈다



값 규칙

DDL 은 그대로, 적재·반영·검증이 같이 쓴다

끝 공백 keep (B02 · B04)	NUL 문자 fail · 적재 전 결정 (A02)
콜레이션 C CI UNIQUE → lower() (B14)	센티널 날짜 keep (A08)

범례 → 변환 kdms plan 이 경고를 내는 규칙

전체 표와 기본값·IDENTITY·계산 컬럼 규칙은 PDF 2쪽부터 · 작업별 덮어쓰기는 config/kdms-rules.yml
규칙이 값을 바꾸면(rtrim · upper · 센티널→NULL) 검증도 같은 규칙으로 원천을 정규화한다

1. 자료형 전체 (기본 규칙 파일에서 생성)

{n} 글자 수(nvarchar 는 바이트/2), {p} 정밀도, {s} 스케일. 여기 없는 타입은 `kdms plan` 이 "규칙 없음" 오류로 멈춘다. 컬럼 하나만 다르게 하려면 `tables.<t>.columns.<c>.type`.

원천(MS-SQL)	대상(PostgreSQL)	비고
<code>bigint</code>	<code>bigint</code>	
<code>int</code>	<code>integer</code>	
<code>smallint</code>	<code>smallint</code>	
<code>tinyint</code>	<code>smallint CHECK (BETWEEN 0 AND 255)</code>	
<code>bit</code>	<code>boolean</code>	A07. 앱 SQL 의 "= 1" 이 많으면 <code>smallint</code> 로
<code>decimal</code>	<code>numeric({p},{s})</code>	
<code>numeric</code>	<code>numeric({p},{s})</code>	
<code>money</code>	<code>numeric(19,4)</code>	A04
<code>smallmoney</code>	<code>numeric(10,4)</code>	
<code>float</code>	<code>double precision</code>	
<code>real</code>	<code>real</code>	
<code>date</code>	<code>date</code>	
<code>time</code>	<code>time({p})</code>	소수 6자리까지 <code>half_up</code> 반올림

원천(MS-SQL)	대상(PostgreSQL)	비고
<code>datetime</code>	<code>timestamp(3)</code>	A05: .997 등 3.33ms 값 그대로
<code>smalldatetime</code>	<code>timestamp(0)</code>	
<code>datetime2</code>	<code>timestamp({p})</code>	소수 6자리까지 half_up 반올림 · A06: 7자리 → 6자리 반올림
<code>datetimeoffset</code>	<code>timestampz({p})</code>	소수 6자리까지 half_up 반올림 · 경고: 오프셋 정보는 버려지고 UTC 시각으로 저장된다
<code>char</code>	<code>char({n})</code>	A10: 채움 공백은 검증에서 RTRIM
<code>nchar</code>	<code>char({n})</code>	
<code>varchar</code>	<code>varchar({n})</code>	(max) → text · A03: CP949 → UTF-8, 문자 수 같고 바이트 증가
<code>nvarchar</code>	<code>varchar({n})</code>	(max) → text
<code>sysname</code>	<code>varchar(128)</code>	
<code>text</code>	<code>text</code>	
<code>ntext</code>	<code>text</code>	
<code>binary</code>	<code>bytea</code>	
<code>varbinary</code>	<code>bytea</code>	
<code>image</code>	<code>bytea</code>	
<code>uniqueidentifier</code>	<code>uuid</code>	경고: 정렬 순서가 달라진다
<code>rowversion</code>	<code>bytea</code>	경고: 앱이 낙관적 잠금에 쓰면 별도 설계(값은 이관 시점 값으로 고정) · sys.types 이름은 timestamp

원천(MS-SQL)	대상(PostgreSQL)	비고
xml	xml	

2. 자료형 밖의 규칙

항목	대상	규칙 키 · 기본값	근거(KIS)
IDENTITY	<code>GENERATED BY DEFAULT AS IDENTITY</code> , 시작·증가값 유지. 정수 타입이 아니면 오류	<code>identity.setval:</code> <code>from_ident_current</code> (전환 때 setval)	A09
SEQUENCE	<code>CREATE SEQUENCE ... AS 타입 START WITH ... [NO] CYCLE</code> . decimal 시퀀스는 bigint + 경고	<code>sequence.setval:</code> <code>from_current_value</code>	B09 · B13
기본값	상수· <code>NEXT VALUE FOR</code> 는 자동. 인자 없는 함수는 표(<code>getdate()</code> → <code>LOCALTIMESTAMP</code> , <code>newid()</code> → <code>gen_random_uuid()</code> , <code>suser_sname()</code> → <code>session_user</code> 등). 그 밖의 식은 오류	<code>defaults.functions</code> , 컬럼별 <code>default</code>	B13
계산 컬럼	<code>GENERATED ALWAYS AS (식) STORED</code> , 적재·반영에서 뺀다. PG 식은 사람이 적는다(없으면 오류)	<code>computed_columns.action:</code> <code>generated_stored</code> , 컬럼별 <code>generated</code>	B12
콜레이션	모든 문자 컬럼에 <code>COLLATE "C"</code>	<code>collation.target: C</code> (C·ko-KR-x-icu·ci)	B01 · B06 · G08
CI UNIQUE	<code>CREATE UNIQUE INDEX ... (lower(col))</code> . FK 가 가리키면 일반 UNIQUE 도	<code>collation.ci_unique: lower_index</code>	B14
NULL 허용 UNIQUE	<code>NULLS NOT DISTINCT</code> (NULL 1건만, MS-SQL 과 같음)	자동	
필터 인덱스	단순 조건만 <code>WHERE</code> 로, 나머지는 경고	자동	

항목	대상	규칙 키 · 기본값	근거(KIS)
이름	소문자. 63바이트 초과는 오류, 겹치는 인덱스 이름은 <code>이름_테이블</code>	<code>identifiers.case: lower</code> , <code>map</code> , <code>schemas</code>	
트리거 · CHECK · 뷰 · SP · 함수 · SYNONYM	옮기지 않고 경고·목록(수작업 전환)	—	B11 · C01~C04
PK 없는 테이블	만들고 경고(전환 때 전체 재적재)	—	plan \$4.6

3. 값 규칙 (DDL 은 바꾸지 않음)

규칙	기본값	선택지	근거(KIS)
끝 공백	<code>keep</code>	keep · rtrim (코드성 컬럼만 rtrim 하려면 컬럼별)	B02 · B04
NUL 문자	<code>fail</code>	fail · strip · replace. <code>--scan</code> 에서 나오면 오류. 3단계 적재 전에 정한다	A02
대소문자	<code>keep</code>	keep · upper · lower	
센티널 날짜(1753-01-01 · 1900-01-01 · 9999-12-31)	<code>keep</code>	keep · null · infinity	A08
소수 초 반올림	<code>half_up</code>	datetime2 · time · datetimeoffset 7자리 → 6자리	A06

값을 바꾸는 규칙을 쓰면 검증도 같은 규칙으로 원천을 정규화한다. 바꾸지 않은 컬럼의 차이는 해시에 그대로 잡힌다(KIS normalization \$0).

4. 시험 원천 KDMS MOCK 결과

- 컬럼 타입은 7개 테이블 55개 컬럼 모두 KIS `mock.sql` 과 같다(Mac 에서 `SourceCatalogIT` · `TargetDdlIT` 통과).
- 작업별 덮어쓰기(`config/kdms-rules.yml`): 계산 컬럼 `dbo.rating.rating_rank` (원천 CI 비교에 맞춰 `upper(rtrim(...))`), `dbo.research_doc.file_ext` 의 PG 식.
- 예상 경고 9건(트리거, uniqueidentifier · rowversion 타입, 함수 기본값 4종, 자동 변환 안 하는 객체), 주의 3건(CI 비교, 끝 공백, CP949 바이트).

- NUL 처리 방식은 아직 정하지 않았다. `--scan` 이 `dbo.issuer.issuer_nm` 에서 막힌다(3단계 전에 결정).